EmojiLang (Phase 1 & Phase 2)

A Visual, Emoji-Based Programming Language

Official Specification by Faiq (23L-0753) and Hasan (23I-0703)

EmojiLang is an educational, emoji-based programming language featuring variables, arrays, half-open slicing, loops, recursive functions, conditionals, and console output.

- **Phase 1**: AST tree-walk interpreter using Flex and Bison in C++17.
- **Phase 2**: Ahead-of-Time (AOT) compiler emitting self-contained LLVM IR, generating native object files via the LLVM TargetMachine API, and linking with Clang to produce standalone executables.

Reserved Tokens and Syntax

Token / Emoji	Name	Description	Example
🔢	Decl	Variable declaration	🔢 i = 0
📥	Assign	Value assignment / array append	📥 result = 📦[🔍(left, i)]
🤔	If	Conditional branch	🤔 📏(arr) <= 1 { ... }
😢	Else	Else branch	😢 { ... }
🔁	Loop	While loop form	🔁 i < 📏(left) { ... }
⚙️ / ⚙️	FuncDef	Function definition	⚙️ merge(left, right) { ... }
↩️ / ↩️	Return	Return statement	↩️ result
📢	Print	Print expression with newline	📢 sorted

Token / Emoji	Name	Description	Example
📦	ArrayLit	Array literal constructor	📦[38, 27, 43, 3, 9, 82, 10]
🔍	Index	Array index access	🔍(arr, i)
📏	Length	Array length	📏(arr)
✂️ / ✂️	Slice	Sub-array [start, end)	✂️(arr, 0, mid)
➕, +	Plus	Addition / Array concatenation	a + b
➖, -	Minus	Subtraction / Unary negation	a - b, -a
✖️, ✖️, *	Multiply	Multiplication	a * b
➗, /	Divide	Integer division	a / b
%	Modulo	Integer modulo	a % b
<, >, <=, >=	Relational	Comparison operators	i < 📏(left)
==, !=, ⚖️, ⚖️	Equality	Equality comparisons	a == b
&&, ||	Logical	Short-circuit logical AND / OR	a && b
{ }	Block	Block delimiters	{ ... }
💬	Comment	Single-line comment	💬 Merge Sort

Architecture & Design

Phase 1: Tree-Walk Interpreter

- **AST (src/ast.h)**: Clean node hierarchy for expressions (NumExpr, VarExpr, BinOpExpr, CallExpr, ArrayLitExpr, IndexExpr, LenExpr, SliceExpr) and statements (DeclStmt, AssignStmt, IfStmt, WhileStmt, FuncDefStmt,

ReturnStmt, PrintStmt, ExprStmt, BlockStmt).

- **Interpreter (src/interp.h, src/interp.cpp)**:
  - Value = std::variant<long long, std::vector<long long>>
  - Lexical scoping stack with RAII scope guards for exception safety.
  - Call boundary returns handled via ReturnSignal.

Phase 2: LLVM Codegen & Native Compilation

- **Code Generator (src/codegen.h, src/codegen.cpp)**:
  - **Type Mapping**:
    - Integers: i64
    - Arrays: { i64, ptr } (representing length and element buffer pointer)
    - Booleans: i64 (0 or 1); branch on icmp ne i64 %val, 0
    - Functions: define i64 @fn(i64 %a) or define { i64, ptr } @fn({ i64, ptr } %arr)
  - **Type Inference**: Iterative static analysis determining whether variables and function signatures handle integers or arrays.
  - **Built-in Self-Contained Runtime**: Emits helper functions directly into the LLVM module (emojilang_print_int, emojilang_print_array, emojilang_slice, emojilang_array_concat, emojilang_array_push, emojilang_array_prepend, emojilang_bounds_check, emojilang_array_eq, emojilang_array_ne).
  - **Target Machine Emission**: Uses llvm::TargetMachine::addPassesToEmitFile to compile directly to native object code (.o), then invokes clang for final linking with standard libc.

File List

```
.
├── CMakeLists.txt          # CMake configuration (Flex, Bison, LLVM)
```

```
├── README.md                # Full documentation (Phase 1 & Phase 2)
├── README.pdf               # Formatted PDF documentation
├── run_tests.sh             # Dual-mode test runner (Interpreter + AOT)
├── docs/
│   └── specification.md     # Language specification & semantics
├── src/
│   ├── ast.h                # Frozen AST node hierarchy
│   ├── codegen.h            # Phase 2 LLVM CodeGen header
│   ├── codegen.cpp          # Phase 2 LLVM CodeGen implementation
│   ├── interp.h             # Phase 1 Interpreter header
│   ├── interp.cpp           # Phase 1 Interpreter implementation
│   ├── lexer.l              # Flex lexer with UTF-8 emoji support
│   ├── parser.y             # Bison LALR(1) parser
│   └── main.cpp             # CLI entry point supporting interpreter, --ir, and --aot
└── tests/
    ├── merge_sort.emoji     # Verbatim Merge Sort from proposal PDF
    ├── insertion_sort.emoji # Insertion Sort test
    ├── loops.emoji          # While-loop iteration test
    ├── recursion.emoji      # Recursive factorial test
    ├── arrays_slice.emoji   # Array slicing test
    └── expected/
        ├── merge_sort.out   # [3, 9, 10, 27, 38, 43, 82]
        ├── insertion_sort.out # [1, 2, 3, 4, 5]
        ├── loops.out        # 0 \n 1 \n 2 \n 3 \n 4
        ├── recursion.out    # 120
        └── arrays_slice.out # [20, 30]
```

Installation & Dependencies

On Ubuntu / Debian systems:

```
sudo apt-get update
sudo apt-get install -y cmake flex bison build-essential llvm-dev clang
```

Required tool versions: - `cmake` >= 3.16 - `flex` >= 2.6 - `bison` >= 3.8 - `g++` >= 9 (C++17) - `llvm-dev` >= 14 (tested with LLVM 18) - `clang` >= 14

Building

```
cmake -B build
cmake --build build
```

The compiled binary will be placed at `./build/emojilang`.

CLI Usage

1. Interpret (Phase 1)

Directly run EmojiLang scripts via the tree-walk interpreter:

```
./build/emojilang tests/merge_sort.emoji
```

Output:

```
[3, 9, 10, 27, 38, 43, 82]
```

2. Print LLVM IR

Emit valid LLVM IR to stdout:

```
./build/emojilang --ir tests/merge_sort.emoji
```

Verify with `llvm-as`:

```
./build/emojilang --ir tests/merge_sort.emoji | llvm-as -o /dev/null
```

3. AOT Compilation to Native Executable

Compile directly to a standalone native binary:

```
./build/emojilang --aot tests/merge_sort.emoji -o /tmp/ms
/tmp/ms
```

Output:

```
[3, 9, 10, 27, 38, 43, 82]
```

Test Suite

Run the automated dual-mode test runner:

```
./run_tests.sh
```

The test runner validates: 1. **Mode 1**: Phase 1 interpreter against expected outputs. 2. **Mode 2**: LLVM IR verification via `llvm-as`, AOT native compilation with Clang, and binary execution against expected outputs.

All 5 core test cases pass 100% in both modes: - `merge_sort.emoji`: `[3, 9, 10, 27, 38, 43, 82]` (verbatim program from PDF) - `insertion_sort.emoji`: `[1, 2, 3, 4, 5]` - `loops.emoji`: prints `0 1 2 3 4` - `recursion.emoji`: `factorial(5)` prints `120` - `arrays_slice.emoji`: ✂️`([10, 20, 30, 40], 1, 3)` prints `[20, 30]`